

클로드코드 Phase 1

누구나 만드는 사람이 된다 — 한 권 요약본

클로드코드로 여는 AI 웹 개발 · Phase 1 (Day 1~5) 압축판

이 요약본은 Day 1~5 다섯 권을 핵심만 남겨 한 편으로 압축한 것입니다. 자세한 예시·화면·과제는 각 [book-dayNN.md](#) 를 보세요.

이 책을 한 문단으로

직군의 경계가 허물어졌다 — 이제 누구나 웹/앱을 만드는 사람이다. 그 도구가 클로드코드(AI 에이전트=비서형, 설치형)다. 만들기 전에 문서(PRD)를 먼저 쓰고, 웹의 구조(프론트엔드·백엔드·DB)를 이해한 뒤, 도구를 잘 부리는 법(컨텍스트·스킬·명령어)을 익히고, Supabase(백엔드+DB)를 MCP로 안전하게 연결해 데이터를 다루며, 마지막에 Git·GitHub·Vercel로 세상에 배포한다. 관통하는 한 문장: 문서 먼저, 코드는 그다음. 그리고 비밀 (`.env`)은 저장소 밖에.

현대 웹 개발 3종 세트 = Claude-Code (개발) + Vercel (프론트 배포) + Supabase (백엔드+DB)

Day 1 · 일하는 방식의 변화와 PRD

핵심 흐름 — 과거엔 기획·디자인·개발·마케팅이 분업했지만, 이제 웹/앱 만들기는 모든 직군의 공통 업무다. 그 도구인 AI를 이해하고, 만들기 전에 PRD를 쓴다.

- **AI 발전 3단계:** 생성형 AI(웹, 생성) → AI 에이전트(설치형, 코딩+내 컴퓨터 조작) → 에이전틱 AI(설치형, 자율 계획·실행)
- **생성형 AI = 챗봇**(물으면 만들어 줌) vs **AI 에이전트 = 비서**(내 컴퓨터를 직접 조작). 클로드코드는 비서다.
- **웹 제작 도구 생태계:** 홈페이지(템플릿) → 프로토타입·MVP(AI채팅+DB) → 프로덕트(+테스트·병렬구현). 클로드코드 하나로 세 단계 전부 커버.
- **클로드코드 실행 = CLI**(명령어 입력). 설치: 내 컴퓨터에 **본체+Node.js**, 프로젝트 폴더에 **플러그인** (`.claude/`).
- **model / effort** 두 손잡이: model=똑똑한 정도(Haiku<Sonnet<Opus), effort=생각의 깊이(low→...→max, Opus만 ultracode).

요소	의미	예
컨텍스트	제한된 기억(토큰)	200k, 1M
모델/에포트	똑똑한 정도(성능)	Opus-4.8
스킬	내장된 프롬프트	frontend-design
훅스	작업 중단/제어	PreToolUse
MCP	외부 연결 도구	supabase-mcp

PRD(제품 요구사항 정의서) = 만들기 전에 누가/무엇을/왜 + 성공 검증 + 최소·확장 기능을 정리. "누가"는 페르소나로 구체화. → ① PRD 만들기 → ② PRD 참고해 홈페이지 만들기(html/css/js).

한 문장 — 무엇을 왜 만들지를 문서로 먼저 정리하고 클로드코드에게 맡겨라.

Day 2 · 웹의 구조와 문서 설계(GDD), 그리고 컨텍스트

핵심 흐름 — 웹의 뒷단(백엔드·DB)과 앞면(프론트엔드)을 이해하고, 설계 문서를 게임(GDD)으로 확장하며, AI의 기억(컨텍스트)을 다스린다.

- **홈페이지 vs 웹 서비스**의 기준 = **DB 시스템 유무**. 홈페이지 = 사용자↔프론트, 웹 서비스 = 사용자↔프론트↔백엔드↔DB (보안·설계·백업 필요).
- **BaaS**(백엔드를 빌려 씀): **Firebase(NoSQL)** vs **Supabase(SQL)**. 백엔드+DB 자리를 **Supabase** 하나로 대체.
- **MPA vs SPA**: MPA=이동마다 새 페이지 다운로드(느림), SPA=처음에 전체 묶음(index.html) 받아 이동 즉시(빠름).
- **기술 진화**: html/css/js (MPA) → **React**(SPA, 첫 페이지만 검색) → **Next**(SPA+, 상세 페이지도 검색=SEO 유리).
- **배포**: 프론트=**Vercel**, 백엔드+DB=**Supabase**.
- **PRD → GDD**(Game Design Document): 만드는 게 게임이면 GDD. 흐름은 동일(문서 먼저 → 코드), 게임 결과물은 **React/Next**.

컨텍스트(working-memory)

- 구성 = 기본 규칙(CLAUDE.md) + 상세 규칙(.claude/rules/) + 지난 대화 + 이번 대화.
- 크기 단위 = **토큰**. 200k≈한글 A4 100장, 1M≈500장. 한글 1글자≈1토큰, 영어 4글자≈1토큰.
- 특징 ① 매 대화마다 직전 컨텍스트 재주입 → 비용 반복. ② 처음·끝은 잘 기억, 중간은 잘 잊음.
- 관리: /clear (초기화, 새 기능 시작) · /compact (압축, 같은 기능 수정·보완).

한 문장 — 뒷단은 빌려 쓰고(Supabase·Vercel), 앞면은 진화한 도구로(React·Next), 설계는 문서 먼저, 기억은 *clear·compact*로 다스려라.

Day 3 · 클로드코드를 잘 부리기 — 명령어와 스킬

핵심 흐름 — 5요소·컨텍스트를 다시 다지고, 클로드코드를 실전에서 다루며, 반복 프롬프트를 ****스킬(자산)****으로 만든다.

- **슬래시 명령어(/)** = 클로드코드의 리모컨. 가장 중요한 건 **컨텍스트 3종**:
`/clear` (초기화) · `/compact` (압축) · `/context` (상태 확인). 그 외 `/init` ([CLAUDE.md](#)), `/effort`, `/model`, `/usage`, `/mcp`, `/skills`, `/doctor` (진단) 등.
- **사용량·요금** (`/usage`): 세션 총비용 + **모델별**(Opus > Haiku). **한도 3종** = 5시간(세션)·1주(전체)·1주(Sonnet). **서브에이전트** 남발 시 **사용량** 급증.
- **터미널 멀티플렉서**: 한 터미널을 여러 창으로 쪼개 여러 에이전트 병렬 실행. **tmux**(Linux/Mac)·**cmux**(Mac, 알람+브라우저 연동)·**wmux**(Windows).
- **플러그인 = 스킬 + 훅스 + MCP**. 전부 `.claude/` 안에 산다.

폴더 구조

```

프로젝트/
├─ .claude/ ─ rules/(상세 규칙) ─ skills/(스킬) ─ hooks/(훅스)
├─ CLAUDE.md (기본 규칙)
├─ .mcp.json (MCP)
└─ html·css·js ...
  
```

스킬(내장된 프롬프트) — 과거엔 매번 “역할 부여 + step by step + 방법론 + `===` + 작업” 프롬프트를 직접 썼다. 이제는 **등록 후 호출**한다.

- **만들기** = `.claude/skills/<이름>/SKILL.md` 에 지침 작성 → **폴더명=스킬명**, 파일 텍스트=프롬프트.
- **호출** = `/스킬이름 + 작업요청` (예: `/banana 홈페이지 만들어줘`). `/reload-skills` 로 재로드.
- **직군별 유명 스킬**: 기획 → `pm-skills`, 디자인 → `frontend-design` · `ui-ux-pro-max`, 개발 → `test-driven-development`.

한 문장 — 명령어로 도구를 부리고, 컨텍스트로 기억을 다스리며, 반복 프롬프트는 스킬로 저장해 자산으로 만들어라.

Day 4 · 서버·데이터·Supabase·MCP

핵심 흐름 — 서버와 접속의 원리를 이해하고, 데이터를 어디에 어떤 모양으로 저장하는지 배우며, Supabase를 만들고 MCP로 안전하게 연결한다.

- **서버 접속 = 호스트 + 포트.** `http://localhost:2763` = 내 컴퓨터의 2763번 프로그램. 포트 범위 0~65535.
- **실서비스 포트:** HTTP=**80**, HTTPS=**443** (URL에서 생략). **접속 = 새로고침** = html 파일 요청/응답.
- **저장 위치:** 로컬스토리지(내 브라우저에만, 기기 종속) vs **외부 DB(Supabase)**(모든 기기 공유).
- **데이터 모양:** **NoSQL**(문서 기반: 컬렉션>다큐먼트>필드, 자유로움 / Firebase) vs **SQL**(표 기반: 테이블>행/열, 안정적 / Supabase=PostgreSQL). 각 레코드는 **id**로 식별.
- **Supabase 제공:** 백엔드 + DB(테이블) + 이미지(스토리지) + 로그인(소셜로그인). 프로젝트 생성 시 조직·이름·DB 비밀번호·지역(**Seoul**) 지정. 테이블 컬럼 = **Name·Type·Default·Primary**(기본키).

MCP (Model Context Protocol) = AI에게 설치해 **외부 서비스에 연결**하는 도구.

- 왜 필요? AI는 **내 컴퓨터**는 조작해도 **외부 서비스**(Supabase·Figma·Notion)엔 직접 못 닿는다.
- MCP 없으면 로그인 제약·통제 불가(위험), 있으면 로그인 설정·제한된 기능으로 통제(안전).
- ⚠️ 사고 사례 **Replit AI가 운영 DB 삭제**("수정 말라 11번" 무시) → 통제 없는 연결의 위험. 대책: 운영/테스트 DB 분리, 권한 최소화(RLS), 백업.

로딩·비밀·연동

- MCP 도구는 보통 **지연로딩**(쓸 때만 로드, 토큰 절약) — 일부는 **즉시로딩**(미리 토큰 차지). `/context` 로 확인.
- `.env` = 비밀번호·접속정보(환경 변수)를 담는 파일. 구조 `이름=값`, `#` 은 주석. **키-밸류** 구조(=NoSQL 필드·SQL 컬럼과 동일 원리). `.gitignore` 로 git 제외.
- **연동 두 축:** **MCP(`.mcp.json` , Access Token)** = 클로드코드가 DB를 관리, **`.env` (URL·anon 키)** = 앱이 DB에 접속. → 로컬스토리지 데이터를 **Supabase로 이전**하면 진짜 웹 서비스.

한 문장 — 서버·데이터의 원리를 이해하고 Supabase로 백엔드를 세우며 MCP로 AI를 외부에 안전하게 연결하라.

Day 5 · 배포 — 세상에 내보내기

핵심 흐름 — 만든 것을 인터넷 주소로 공개(배포)한다. 자동 배포의 원리와 비밀 관리를 익히고, Git·GitHub·Vercel로 실제로 띄운다.

- **배포 3준비물:** ① 도메인 주소(가비아·고대디) ② 24시간 켜진 서버 ③ 외부 접속 네트워크 설정.
- **어디에 올릴까**(오른쪽일수록 편리): 내 컴퓨터 → **클라우드**(AWS·GCP·Azure, 24시간 해결) → **배포 플랫폼**

(코드만 올리면 끝, 트래픽 ↑ = 요금 ↑). 프론트=Vercel·Netlify, 백엔드=Render·Railway.

- **백엔드 만들기 vs 빌리기**: Render/Railway=내가 만든 백엔드 배포, Supabase=이미 만들어진 백엔드 갖다쓰기.
- **배포 구성**: 홈페이지=Vercel만, 웹 서비스=Vercel + Supabase.

자동 배포 & 비밀

[내 컴퓨터] -빌드→업로드→ [코드 저장소(GitHub)] -감지·자동전송→ [Vercel]
└─── 연결 1회 ───┘

- **빌드** = 문법검사·압축·최적화("빌드 해줘"). **에러**(안 돌아감) ≠ **버그**(돌아가는데 결과가 틀림).
- 저장소 ↔ Vercel **최초 1회 연결** → 이후 자동(CI/CD). 저장소는 **GitHub**(대안 GitLab·Bitbucket).
- ⚠ **.env** 는 **GitHub에 올리지 않는다**(.gitignore) → **Vercel에 직접 등록**.
- 사고 **티빙 개인정보 유출(~1300만 명)**: 진짜 원인은 DB가 아니라 **GitHub에 노출된 자격증명·AWS 키 하드코딩**. 교훈: 비밀은 **.env** 로 분리, **.gitignore** 로 제외, 키 주기적 교체.

Git·GitHub & 실전

- **Git**(내 컴퓨터 도구, 설치) vs **GitHub**(인터넷 저장소, 가입). 저장 단위 = **레포지토리**(Public/Private).
- **커밋 → 푸쉬**: 커밋= **.git** 에 저장(로컬), 푸쉬= **.git** 을 GitHub에 업로드. ("깃 커밋 해줘" → "깃 푸쉬 해줘")
- **리모트** = 내 **.git** 이 바라볼 GitHub 주소(...git). **Vercel 무료 도메인**(...vercel.app) 자동 부여.
- 실전: 클로드코드에게 "깃 초기화 → 리모트 셋팅 → **.env** 분리 → **.gitignore** 설정" 후 "**빌드 → 커밋 → 푸쉬**".

한 문장 — 빌드해서 저장소에 올리면 Vercel이 자동 배포한다. 비밀(**.env**)만은 저장소 밖에 두어라.

전권 관통 원칙

1. 문서 먼저, 코드는 그다음 — PRD/GDD를 쓰고 나서 만든다.
2. **AI 에이전트 = 비서** — 5요소(컨텍스트·모델/에포트·스킬·혹스·MCP)로 이해하고 다룬다.
3. **컨텍스트를 다스려라** — 재주입 비용·중간 망각 → **/clear · /compact** .
4. **반복은 자산으로** — 프롬프트를 스킬로 등록해 재사용.
5. **외부 연결은 통제 아래** — MCP로 로그인·권한을 제한(Replit 사고 교훈).
6. **비밀은 저장소 밖에** — **.env** 는 **.gitignore** 로 제외, 배포처엔 직접 등록(티빙 사태 교훈).
7. **한 번 연결하면 자동** — GitHub ↔ Vercel CI/CD, 커밋 → 푸쉬만으로 배포.

전권 핵심 용어 사전

용어	한 줄 정의
PRD / GDD	만들기 전 요구사항 문서(누가/무엇을/왜) · 게임용 버전
컨텍스트 / 토큰	AI의 작업 기억 · 그 크기 단위(한글 1자≈1토큰)
스킬 / 훅스 / MCP	내장 프롬프트 · 작업 제어 · 외부 연결 (플러그인 3요소)
홈페이지 / 웹 서비스	DB 없음 · DB 있음(프론트+백엔드+DB)
BaaS	백엔드+DB를 빌려 쓰는 서비스(Firebase=NoSQL, Supabase=SQL)
MPA / SPA	이동마다 다운로드(느림) · 처음에 한 번(빠름)
React / Next	SPA(첫 페이지만 검색) · SPA+SEO(상세도 검색)
호스트 / 포트	어느 컴퓨터 · 그 안의 몇 번 프로그램(HTTP 80/HTTPS 443)
로컬스토리지 / 외부 DB	내 브라우저에만 · 모든 기기 공유(Supabase)
NoSQL / SQL	문서(컬렉션>다큐먼트>필드) · 표(테이블>행/열)
<code>.env</code> / 환경변수	비밀·접속정보를 담는 파일 · 이름=값 키-밸류
빌드 / 에러 / 버그	문법검사·압축·최적화 · 안 돌아감 · 결과가 틀림
Git / GitHub	내 컴퓨터 도구(설치) · 인터넷 저장소(가입)
커밋 / 푸쉬	<code>.git</code> 에 저장(로컬) · GitHub에 업로드(원격)
리모트 / CI/CD	<code>.git</code> 이 바라볼 GitHub 주소 · 올리면 자동 배포되는 흐름
Vercel / Supabase	프론트 배포 플랫폼 · 백엔드+DB(BaaS)

Phase 1 졸업 체크리스트

- ☐ PRD를 먼저 쓰고 코드는 나중에 만드는 흐름이 몸에 붙었다
- ☐ Claude-Code + Vercel + Supabase 3종의 역할 분담을 안다
- ☐ 컨텍스트를 `/clear` · `/compact` 로 관리할 수 있다
- ☐ 클로드코드에 Supabase MCP를 연결해 테이블을 다룰 수 있다
- ☐ 소개 페이지가 배포되어 누구나 접속할 수 있다
- ☐ 비밀 값(`.env`)은 저장소 밖에서 관리한다

다음 단계 → **Phase 2 · 미니 노션 (Day 6-16)** — 혼자 만들던 방식에서, 기획 → 디자인 → 개발을 핸드오프로 잇는 실무형 협업 구조로 확장합니다.

